# Practice Exam 2 - Programming for Minor Computational Science

Name:

.....

**Log in:**

You will make this exam using the online editor. To access it:

- Go to **mcs.proglab.nl**.
- Enter the exam code: **abc123**
- Enter your **student number**.
- Keep your student card or ID on your desk.

**Goal:**

- This digital exam consists of 5 assignments where you will write short Python programs.
- You need at least 3 assignments correct in order to pass the exam.
- You will be **evaluated solely on the correctness** of your solutions. Code design, comments, and style are not important, so you do not need to worry about them.
- The editor has a Checkpy button that allows you to verify your solutions during the exam for your personal use. Note that Checkpy results will not influence your actual grade, as all grading is done manually. So, even if your solutions do not pass Checkpy, they may still be accepted during the grading process.

**Constraints:**

- You may use the course website as a resource (open mcs.proglab.nl again in a new tab after logging in).
- You may use a printed version of the cheatsheet during the exam.
- Do not use external modules such as `numpy`, `csv`, or others unless the assignment specifically says you can.

**Submit:**

When you have completed all assignments:

- use the **submit button** on the website to ensure your final edits are committed,
- **please fill out the evaluation** (link in submit window), this helps us a lot to improve the course,
- close the browser,
- hand in this exam sheet **with your name on it**.

## Assignment 1: Count Words

Write a function `count_words(text)` that counts how often each word appears in `text`, and returns the result as a dictionary. Capitals and punctuation do not matter, so `The` and `the.` count as the same word.

You can use the function `tokenize(text)` below to convert the text into a list of words:

```
def tokenize(text):
    return [word.strip(" ,\n.);(!?)'").lower() for word in text.split(' ')]
```

**Example Usage**

```
print(count_words("The cat and the hat. The end!"))
```

**Expected Output**

```
{'the': 3, 'cat': 1, 'and': 1, 'hat': 1, 'end': 1}
```

## Assignment 2: Group by Length

Write a function `group_by_length(words)` that takes a list of words and groups them by their length. The function returns a dictionary in which each key is a word length, and the value is a list of all words with that length, in the order in which they appear in `words`.

**Example Usage**

```
print(group_by_length(["hi", "sun", "to", "sky", "a"]))
```

**Expected Output**

```
{2: ['hi', 'to'], 3: ['sun', 'sky'], 1: ['a']}
```

## Assignment 3: Warmer Days

Write a function `warmer_days(temperatures)` that takes a list of daily temperatures and returns the positions of all days that were warmer than the day before. Positions start at 0, so the first day is never included.

**Example Usage**

```
temperatures = [12, 14, 13, 15, 17, 16, 16, 18]
print(warmer_days(temperatures))
```

**Expected Output**

```
[1, 3, 4, 7]
```

## Assignment 4: Decompress

A simple way to compress a text is to store every piece of text (a *token*) only once, together with its positions in the text. A position tells in which order the tokens appear: position 1 is the first token, position 2 is the second token, and so on. For example, the text `banana` consists of the tokens `ba`, `na` and `na`, so it can be stored as:

```
{"ba": [1], "na": [2, 3]}
```

Write a function `decompress(tokens)` that takes such a dictionary and returns the original text.

**Example Usage**

```
tokens = {"How": [1], " ": [2, 4, 6, 9, 11, 14, 16, 18, 20, 23, 26, 28], "much": [3],
          "wood": [5, 12, 21, 29], "w": [7], "ould": [8, 25], "a": [10, 19],
          "chuck": [13, 15, 22, 27], "if": [17], "c": [24], "?": [30]}
print(decompress(tokens))
```

**Expected Output**

```
How much wood would a woodchuck chuck if a woodchuck could chuck wood?
```

## Assignment 5: Draw

A picture made of characters can be stored as a dictionary. Each key is a character, and the value is a list of `(row, column)` tuples: the positions where that character appears in the picture. Rows and columns start at 0, so `(0, 0)` is the top left corner.

Write a function `draw(pixels)` that takes such a dictionary and returns the picture as a string, with one line per row.

**Example Usage**

```
pixels = {'.': [(0, 0), (0, 2), (0, 4), (1, 1), (1, 2), (1, 3), (2, 0), (2, 4)],
          '#': [(0, 1), (0, 3), (1, 0), (1, 4), (2, 1), (2, 2), (2, 3)]}
print(draw(pixels))
```

**Expected Output**

```
.#.#.
#...#
.###.
```